

combinations

Generate all element combinations of arrays
Since R2023a

Syntax

```
T = combinations(A1,...,An)
```

Description

T = combinations(A1,...,An) generates all element combinations of input arrays A1,...,An, where each row of the output table T is a combination. These combinations are the same as the Cartesian product of *n* sets of elements where the *n* sets are represented by the input arrays A1,...,An. [example](#)

- The number of rows in T is the product of the number of elements in each input array. The number of rows is the same as the number of combinations.
- The number of variables in T is the same as the number of input arrays.
- The data type of each variable in T is the same as the data type of the corresponding input array.

Examples

[collapse all](#)

Vector Combinations

Generate all element combinations of two vectors. Each row of the output table T is a combination with the first element coming from the first vector and the second element coming from the second vector. The variable Var1 contains elements from the first vector, and the variable Var2 contains elements from the second vector.

[Open in MATLAB Online](#)

[Copy Command](#)

```
T = combinations([1 8 6],[9 3 2])
```

 Get ▼

T=9x2 table

Var1	Var2
1	9
1	3
1	2
8	9
8	3
8	2
6	9
6	3
6	2

Array Combinations

Generate all element combinations of two arrays. The combinations function reshapes each input into a column vector before generating the element combinations.

[Open in MATLAB Online](#)
[Copy Command](#)

```
T = combinations([7 3;2 6],[9 1 4])
```

[Get](#) ▼

T=12×2 table

Var1	Var2
——	——
7	9
7	1
7	4
2	9
2	1
2	4
3	9
3	1
3	4
6	9
6	1
6	4

String Array Combinations

Generate all element combinations of three string arrays. The names of variables in the output table are based on the names of the string arrays.

[Open in MATLAB Online](#)
[Copy Command](#)

```
ID = ["A" "B" "C"];
color = ["red" "blue" "green"];
sz = ["small" "large"];
```

```
T = combinations(ID,color,sz)
```

[Get](#) ▼

T=18×3 table

ID	color	sz
——	——	——
"A"	"red"	"small"
"A"	"red"	"large"
"A"	"blue"	"small"

```

"A"    "blue"    "large"
"A"    "green"   "small"
"A"    "green"   "large"
"B"    "red"     "small"
"B"    "red"     "large"
"B"    "blue"    "small"
"B"    "blue"    "large"
"B"    "green"   "small"
"B"    "green"   "large"
"C"    "red"     "small"
"C"    "red"     "large"
"C"    "blue"    "small"
"C"    "blue"    "large"
:
```

Create data labels by combining the elements across each combination. Extract the contents of the table T using curly braces {}, and combine elements across rows using join.

```
labels = join(T{:, :}, "_")
```

 Get ▾

```
labels = 18x1 string
```

```

"A_red_small"
"A_red_large"
"A_blue_small"
"A_blue_large"
"A_green_small"
"A_green_large"
"B_red_small"
"B_red_large"
"B_blue_small"
"B_blue_large"
"B_green_small"
"B_green_large"
"C_red_small"
"C_red_large"
"C_blue_small"
"C_blue_large"
"C_green_small"
"C_green_large"
```

Inputs of Different Data Types

Generate all element combinations of inputs of different sizes and data types.

The number of rows in the output table T is equal to the product of numel(A1), numel(A2) and numel(A3). The data type of each variable in the output table T is the same as the data type of the corresponding input.

Open in MATLAB
Online

 Copy Command

```

A1 = [0 1; 2 3];
A2 = ["a" "b" "c"];
```

 Get ▾

```
A3 = categorical(["x" "y"]);
T = combinations(A1,A2,A3)
```

T=24×3 table

A1	A2	A3
0	"a"	x
0	"a"	y
0	"b"	x
0	"b"	y
0	"c"	x
0	"c"	y
2	"a"	x
2	"a"	y
2	"b"	x
2	"b"	y
2	"c"	x
2	"c"	y
1	"a"	x
1	"a"	y
1	"b"	x
1	"b"	y
:		

Sweep Parameter Combinations Using rowfun

You can generate a table of element combinations for multiple parameters and use the [rowfun](#) function to apply a function to each row. Use the combinations and rowfun functions together to efficiently compute and organize the result for each combination.

Generate a table that contains all possible combinations of the Percentage and Method parameters.

[Open in MATLAB Online](#)

[Copy Command](#)

```
Percentage = [25; 75];
Method = ["midpoint"; "inclusive"; "exclusive"];
T = combinations(Percentage,Method)
```

[Get](#) ▼

T=6×2 table

Percentage	Method
25	"midpoint"
25	"inclusive"
25	"exclusive"
75	"midpoint"
75	"inclusive"
75	"exclusive"

Use the rowfun function to apply the prctile function for each row of the table. rowfun returns a table that contains the resulting percentile for every parameter combination.

```
A = rand(13,1);
T2 = rowfun(@(p,m) prctile(A,p,Method=m),T, ...
OutputVariableNames="Percentile");
```

 Get ▾

Append the results to the table of parameter combinations so you can see which parameter values produced each output.

```
T3 = [T T2]
```

 Get ▾

T3=6×3 table

Percentage	Method	Percentile
25	"midpoint"	0.24828
25	"inclusive"	0.2785
25	"exclusive"	0.21806
75	"midpoint"	0.95725
75	"inclusive"	0.95717
75	"exclusive"	0.95734

Input Arguments

[collapse all](#)

A1, ..., An — Input arrays (as separate arguments)

scalars | vectors | matrices | multidimensional arrays

Input arrays, specified as a comma-separated list of scalars, vectors, matrices, or multidimensional arrays. The input arrays can have different sizes and data types.

Data Types: single | double | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64 | logical | char | string | struct | datetime | duration | calendarDuration | categorical | cell

Complex Number Support: Yes

Tips

• Categorical arrays

If your input arrays are categorical arrays, you might want to produce another categorical array whose categories are all combinations of the inputs. Because the combinations function returns all combinations in a table, consider using the .* operator instead.

For example, create two categorical arrays and call the .* operator.

```
colors = categorical(["blue" "red" "green" "black"]);
signs = categorical(["+" "-" "-" "+"]);
C = colors .* signs
```

C =

1×4 categorical array

blue + red - green - black +

The result is a categorical array whose categories are all combinations of the categories of the inputs. List the categories using the `categories` function.

```
categories(C)
```

```
ans =
```

```
8x1 cell array
```

```
{'black '+'}  
{'black -'}  
{'blue '+' }  
{'blue -' }  
{'green '+'}  
{'green -'}  
{'red '+' }  
{'red -' }
```

Version History

Introduced in R2023a

See Also

[ndgrid](#) | [meshgrid](#) | [perms](#) | [nchoosek](#) | [repelem](#) | [repmat](#) | [rowfun](#)

Topics

[Produce All Combinations of Categories from Two Categorical Arrays](#)

YOU MIGHT ALSO BE INTERESTED IN

[Integrating MATLAB with Python](#)

Use MATLAB® as a computational engine in Python®, or use Python libraries and models in MATLAB.

[Learn more](#)